

TP n°1

Programmation à objets avancé : langage C++

Bases du langage

L'objectif de ce TP est d'appréhender les notions de base du développement objet en C++.

La documentation officielle du langage et de la librairie standard du C++ peut être consultée à cette adresse : <https://en.cppreference.com/>.

1 Ma première classe

L'objectif de cet exercice est de créer une première classe qui va nous permettre d'étudier les différents éléments constitutifs d'une classe.

Nous allons utiliser une structure permettant de gérer un vecteur de nombres flottants de dimension variables au cours du temps.

1.1 Instanciation, constructeur, destructeur

Nous allons commencer par créer une première version de la classe **Vector** très simple permettant d'illustrer les concepts de base. Le diagramme UML définissant cette première classe est fourni dans la figure 1.

Pour chacune des fonctions que vous créerez, pensez à ajouter une sortie textuelle sur la console afin de pouvoir tracer simplement les appels de fonctions successives (notamment les constructeurs et destructeur).

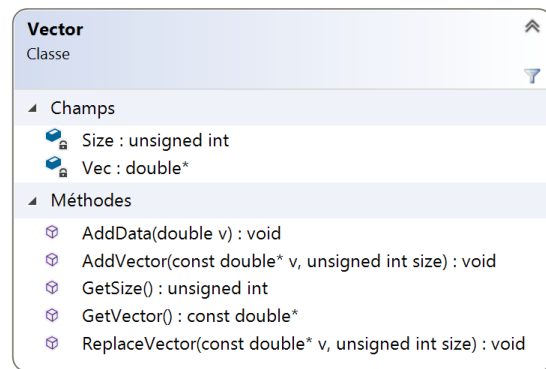


FIGURE 1 – Diagramme UML de la classe **Vector**.

QUESTION 1. Créez un nouveau fichier *header* dans lequel vous allez écrire le prototype de la classe **Vector** correspondant à son diagramme UML. Dans un premier temps, ne prenez en compte que les attributs, les méthodes seront progressivement introduites dans les questions suivantes.

QUESTION 2. Créez ensuite un nouveau fichier *source* dans lequel vous implémenterez les méthodes définies dans la classe **Vector**. Implémentez les méthodes **GetSize** et **GetVector** qui renvoient respectivement la taille (attribut **Size**) et le contenu du tableau (attribut **Vec**). Attention, il ne doit pas être possible pour l'utilisateur de nos fonctions de modifier le contenu des éléments retournés.

QUESTION 3. Créez un constructeur par défaut qui initialise la taille du vecteur à 0.

QUESTION 4. Créez un destructeur pour notre classe.

QUESTION 5. Créez un nouveau *source* (nommé *main.cpp* par exemple) qui contiendra le point d'entrée de votre programme (la fonction `main`). Dans le `main`, instanciez un premier vecteur (`vec1`). Vérifiez avec le débogueur quelles sont les différentes fonctions qui sont appelées lors de la création de l'objet `vec1` jusqu'à sa destruction à la fin de la fonction `main`.

QUESTION 6. Depuis le `main`, essayez de modifier l'attribut `Size`. Que ce passe-t-il ? Pourquoi ?

1.2 Constructeur par défaut, constructeur par recopie

QUESTION 1. Implémentez les fonctions `AddData` (ajout d'un élément supplémentaire dans le vecteur), `AddVector` (ajout d'un vecteur à la suite du vecteur courant) et `ReplaceVector` (remplacement du vecteur interne par un nouveau vecteur) et testez-les.

QUESTION 2. Mettez le constructeur par défaut en commentaire et vérifiez que votre code compile toujours puis exécutez-le. Que se passe-t-il ?

QUESTION 3. Remplacez votre `main` initial par celui fourni dans le listing 1. En laissant le constructeur par défaut en commentaire, ajoutez un constructeur prenant en paramètre un tableau de valeurs et sa taille afin d'initialiser le vecteur à une valeur non nulle. Compilez et exécutez.

Listing 1 – Première version du `main`

```
void PrintVector(const Vector &v)
{
    printf("[ ");
    for (unsigned int i=0; i<v.GetSize(); ++i)
    {
        if (i!=0) printf(", ");
        printf("%f", v.GetVector()[i]);
    }
    printf("]\n");
}

int main()
{
    double tab1[5]={1.0, 2.0, 3.0, 4.0, 5.0};
    double tab2[2]={9.0, 10.0};
    Vector v1(tab1, 5);

    v1.AddVector(tab2, 2);
    printf("v1=");
    PrintVector(v1);

    return 0;
}
```

QUESTION 4. Ajoutez un nouvel objet faisant appel au constructeur sans paramètre (`Vector v2;`) et compilez. Comparez le résultat avec celui obtenu dans la question 2.

QUESTION 5. Enlevez les commentaires autour de votre constructeur par défaut et remplacez le `main` par celui fourni dans le listing 2. Compilez et exécutez. Que se passe-t-il ?

QUESTION 6. Corrigez votre code de manière à ce que le comportement soit celui attendu.

1.3 Opérateurs *new* et *delete*

QUESTION 1. Reprenez le code du **main** précédent en remplaçant la création des objets sur la pile par le compilateur par la création des objets sur le tas (déclaration des pointeurs au lieu des objets : **Vector *v1**).

QUESTION 2. Créez un tableau de 10 objets **Vector** que vous initialiserez dans un premier temps avec le constructeur par défaut et dans un deuxième temps en utilisant un tableau d'initialisation différent pour chaque objet. N'oubliez pas de détruire tous les objets créés !

Listing 2 – Deuxième version du main

```
#define DebugVector(v) {printf("#v="); PrintVector(v);}

void PrintVector(const Vector &v)
{
    printf("[");
    for (unsigned int i=0; i<v.GetSize(); ++i)
    {
        if (i!=0) printf(",");
        printf("%f", v.GetVector()[i]);
    }
    printf("]\n");
}

int main()
{
    double tab1[5]={1.0, 2.0, 3.0, 4.0, 5.0};
    double tab2[3]={6.0, 7.0, 8.0};
    double tab3[2]={9.0, 10.0};
    Vector v1(tab1, 5);
    Vector v2;

    v2.AddVector(tab2, 3);
    DebugVector(v1);
    DebugVector(v2);
    /*
    Renvoi sur le terminal :
    v1=[1.0, 2.0, 3.0, 4.0, 5.0]
    v2=[6.0, 7.0, 8.0]
    */

    Vector v3(v1);
    v1.AddData(11);
    v2.ReplaceVector(tab3,2);
    DebugVector(v1);
    DebugVector(v2);
    DebugVector(v3);
    /*
    Renvoi sur le terminal :
    v1=[1.0, 2.0, 3.0, 4.0, 5.0, 11.0]
    v2=[9.0, 10.0]
    v3=[1.0, 2.0, 3.0, 4.0, 5.0]
    */

    return 0;
}
```